

Basics of Python & OOP

Assignment 1

Deadline: 2 June 2026 (EOD)

Instructions

- Attempt all questions.
 - Write clean and readable code.
 - Explain your logic wherever necessary, via comments
 - You may create helper functions wherever required.
-

Question 1: Student Performance Analyzer (Easy)

Write a program that:

1. Takes marks of 10 students as input.
 2. Stores them in a list.
 3. Calculates:
 - Highest marks
 - Lowest marks
 - Average marks
 - Number of students scoring above average
 4. Assign grades:
 - A : ≥ 90
 - B : ≥ 75
 - C : ≥ 60
 - D : otherwise
 5. Display the grade distribution using a dictionary.
-

Question 2: Color Grid Processor (Medium)

A Rubik's Cube face is represented as:

```
face = [  
    ["R", "R", "G"],  
    ["W", "R", "G"],  
    ["B", "B", "Y"]  
]
```

Write a program that:

1. Prints the grid neatly.
 2. Counts the frequency of every color.
 3. Replaces every occurrence of:

 R -> X
 4. Determines the most common color.
 5. Checks whether any row contains all identical colors.
-

Question 3: Text Analytics Tool (Medium)

Given a sentence entered by the user:

1. Count total characters.
2. Count total words.
3. Count vowels.
4. Count frequency of every character using a dictionary.
5. Find the most frequently occurring character.
6. Check whether the sentence is a palindrome after removing spaces and converting to lowercase.

Example:

Input:

Never Odd Or Even

Output:

Palindrome : True

Question 4: Functions, Scope and Mutability Challenge (Hard)

Consider the following code:

```
def modify_number(x):  
    x += 10
```

```
def modify_list(lst):  
    lst.append(100)
```

```
num = 5  
values = [1,2,3]
```

```
modify_number(num)  
modify_list(values)
```

```
print(num)  
print(values)
```

Answer the following:

1. Predict the output.
 2. Explain why the outputs differ.
 3. Explain mutable and immutable objects.
 4. Write your own examples demonstrating:
 - One immutable type
 - One mutable type
 5. Create a function that safely modifies a list without changing the original list.
-

Mini Project (50 Marks): Speedcubing Club Management System

The Speedcubing Club wants a software system that can be reused by future coordinators to manage members, attendance and events.

Class: Member

Each member should store:

- Name
- Roll Number
- Branch
- Year
- Cubing Average

Methods:

- display_details()
- update_average()

Class: Event

Each event should store:

- Event Name
- Date
- Venue
- List of Registered Members

Methods:

- register_member()
- remove_member()
- display_participants()

Class: Club

The Club class should:

- Store all members.
- Store all events.
- Add a member.
- Remove a member.
- Search for a member using roll number.
- Create an event.
- Display all events.
- Display all members.
- Display the top 3 fastest cubers in the club.
- Find average cubing time of all members.
- Count members branch-wise.

Bonus Features

Implement any two:

1. Attendance tracking for workshops.
2. Member leaderboard based on cubing averages.
3. Event-wise participant statistics.
4. Search members by branch.

5. Export member information in a readable text format.

Reflection Questions

Answer the following:

1. Where did you use encapsulation?
2. Which classes interact with each other and why?
3. Which variables are instance variables?
4. Which variables could be class variables?
5. What modifications would be required to reuse your system for next year?